Task 1 - Index Performance Analysis

1. Calculate Selectivity:

Selectivity represents the fraction of the dataset that matches the query conditions. The formula for selectivity in this case is:

Selectivity = (product count x store count x time count) / (total products x total stores x total time period)

Given:

- Products: 1 (iPhone 14)
- Stores: 200 (California stores)
- Time: 90 (Q1 2024)
- Total Products: 50,000
- Total Stores: 2,000
- Total Time periods: 1,825 days

Selectivity = (1 x 200 x 90)/(50000 x 2000x 1825) = 9.86 x 10 ^ (-8)

This means the query will return about **0.0000986** (0.00986%) of the entire dataset.

2. Calculate Data to Read:

To calculate how much data needs to be read, we multiply the size of the data warehouse by the selectivity:

Data to Read = Total data size x selectivity

Given:

- Total Data Size: 50 GB
- Selectivity: (9.86 \times 10^-5)

Data to Read = 50GB x 9.86 x 10 ^ -5 = 4.93 x 10 ^ (-3) B = 4.93 MB

So, the query will read approximately **4.93 MB** of data.

3. Estimate Time: Full Scan vs. With Indexes:

- **Full Scan**: If we perform a full scan of the dataset, we need to read the entire 50 GB. At a read rate of 100 MB/s, the time taken for a full scan would be:

  Time for full scan = 50000/100 = 500 seconds

- **With Indexes**: Using indexes, the data to be read is reduced to 4.93 MB. The time taken to read this data at 100 MB/s would be:

  Time with indexes = 4.93/100 = 0.0493 seconds

So, **with indexes**, the query will take approximately **0.05 seconds** to retrieve the required data.

4. Design 3 Optimal Indexes:

For the given query "Sales of iPhone 14 in California during Q1 2024", the most useful indexes would be on the columns that are heavily filtered by the query:

1. **Composite Index on Product and Store**:
    - This index helps quickly narrow down the rows that match the condition on Product (iPhone 14) and Store (California stores).
    - **Index**: (Product, Store)
2. **Composite Index on Product and Time**:
    - This index helps quickly retrieve the sales of iPhone 14 for the given time range (Q1 2024).
    - **Index**: (Product, Time)
3. **Composite Index on Store and Time**:

- This index will help filter the dataset by store location and the specific time range, helping with performance when these two columns are combined in the query.

- **Index**: (Store, Time)

These indexes would minimize the amount of data scanned, allowing for efficient data retrieval.

Task 2 - R-Tree Implementation

1. Build R-Tree with M=3, m=2:

Given the following products with their coordinates:

- P1: (10, 20) - Electronics
- P2: (15, 25) - Electronics
- P3: (12, 22) - Electronics
- P4: (50, 60) - Clothing
- P5: (55, 65) - Clothing
- P6: (52, 62) - Clothing

R-Tree Node Structure:

- Each internal node can hold up to 3 entries (M=3).
- Each leaf node can hold up to 3 entries, but must have at least 2 entries (m=2).
- An entry is a pair: (MBR, pointer to child or data).

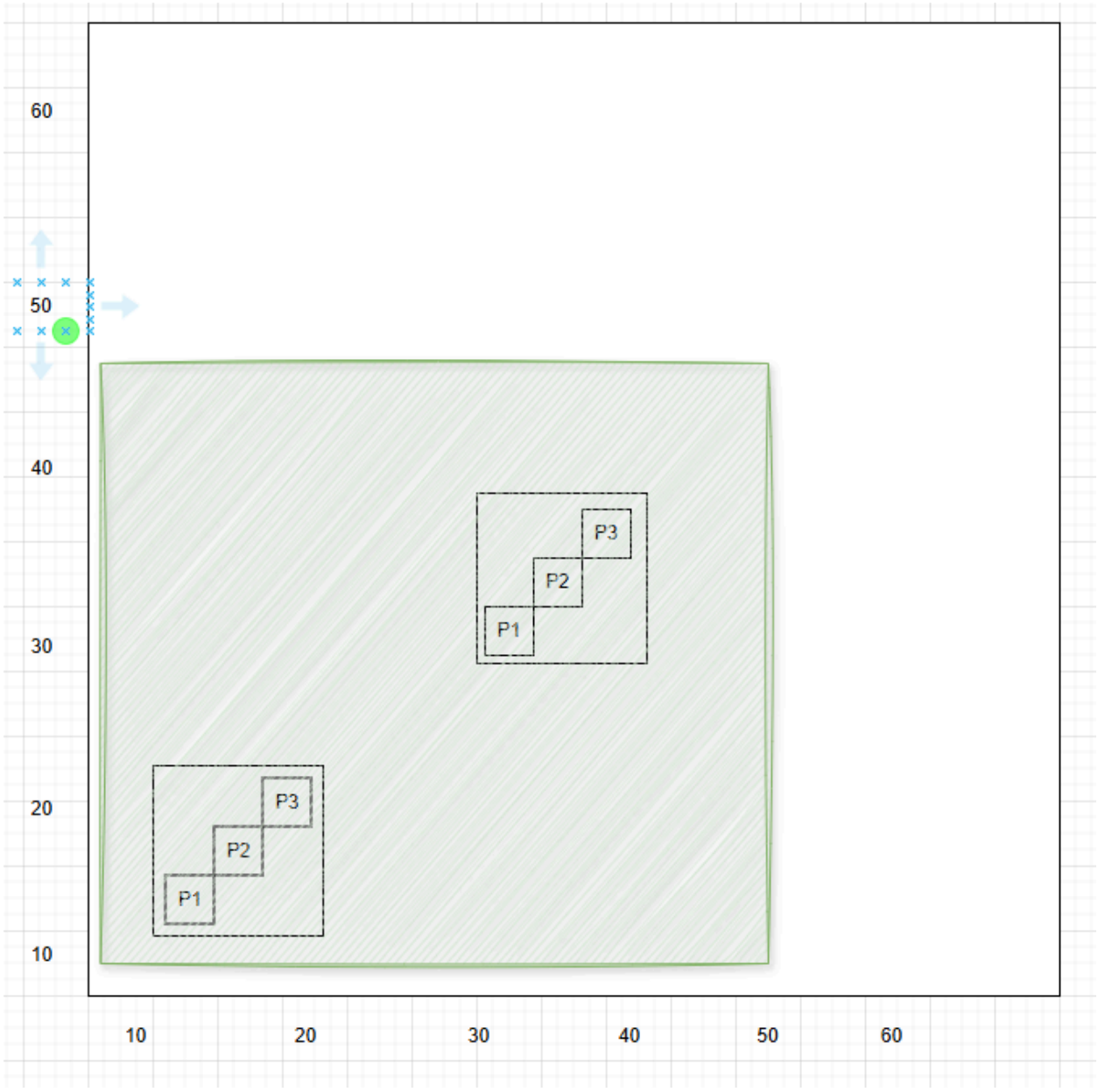
Building the R-Tree:

1. **Initial Leaf Nodes**:
    - **Leaf 1**: P1, P2, P3 → MBR: [(10, 20), (15, 25)]
    - **Leaf 2**: P4, P5, P6 → MBR: [(50, 60), (55, 65)]

2. **Internal Nodes**:

    - **Internal Node 1**: Points to Leaf 1 and Leaf 2 → MBR: [(10, 20), (55, 65)]

Now the R-Tree structure looks like this:



2. SEARCH: Find all products in the rectangle [(10, 15), (55, 70)]

- Start with the root node.

- The query rectangle overlaps with the MBR of both Leaf Node 1 and Leaf Node 2.
    - **Leaf Node 1**: Overlaps with the search rectangle, so we need to check the individual products (P1, P2, P3).
    - **Leaf Node 2**: Overlaps with the search rectangle, so we need to check the individual products (P4, P5, P6).

Visited Nodes: Root Node, Leaf Node 1, Leaf Node 2

Pruned Nodes: None (both leaf nodes are fully checked)

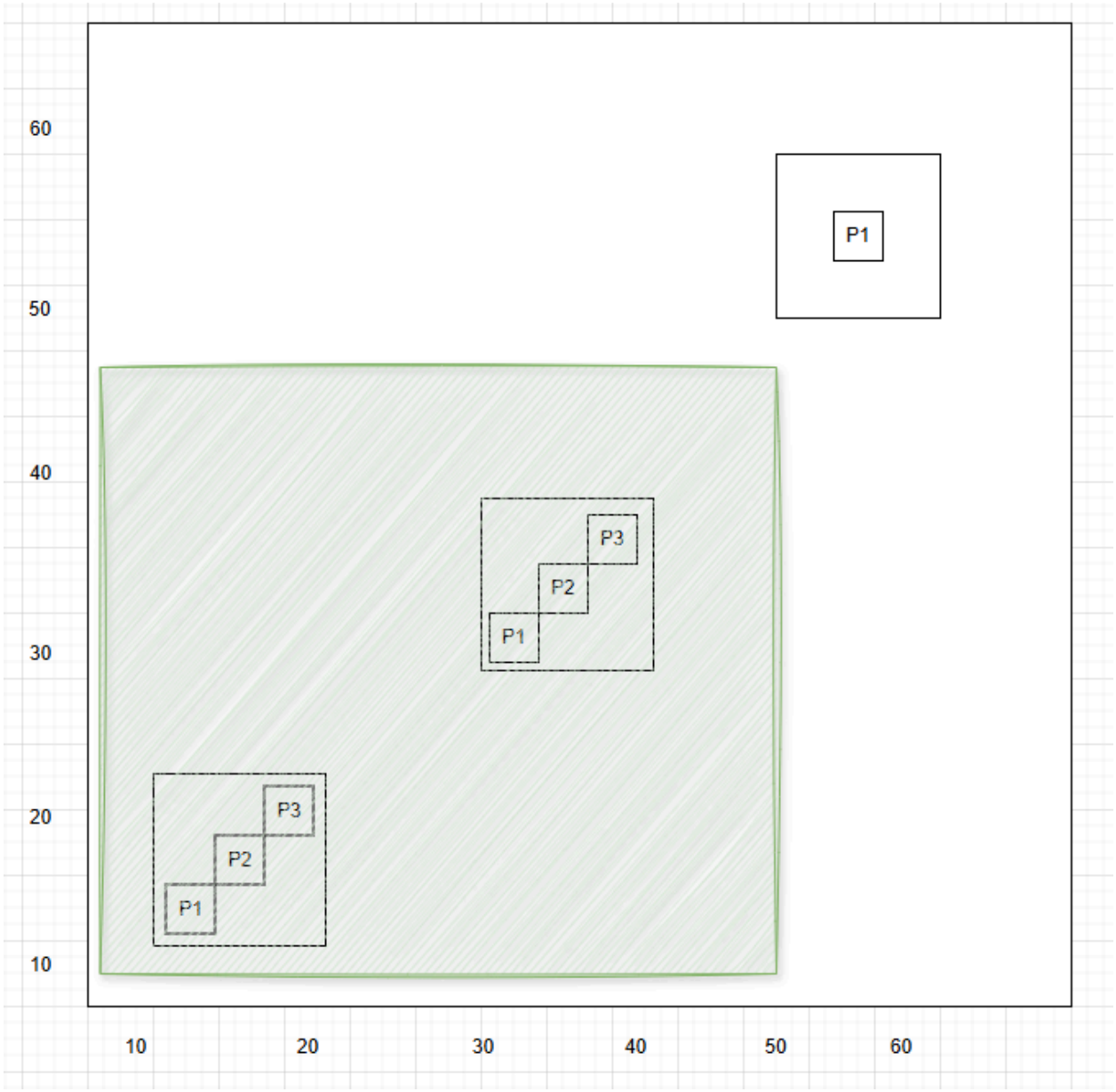
3. INSERT: Add P7: (58, 68)

- P7: (58, 68) is inserted into the tree.
- It goes into **Leaf Node 2**, because it overlaps with the existing MBR of Leaf Node 2.
- Now, **Leaf Node 2** holds 4 entries, which exceeds the maximum of 3.
- **Overflow Handling**: A split occurs, and the entries in Leaf Node 2 are split between two leaf nodes.

New structure:

- **Leaf Node 2**: P4, P5 → MBR: [(50, 60), (55, 65)]
- **Leaf Node 3**: P6, P7 → MBR: [(52, 62), (58, 68)]

Now the structure looks like:



4. Nodes Visited vs Total Nodes

- **Total nodes**: 3 (1 root, 2 leaf nodes).
- **Nodes visited**: 3 (root node, leaf node 1, and leaf node 2 or 3, depending on the split).

Task 4 - R-Tree vs B-Tree Comparison

Scenario A: Single Dimension Search

Query:

SELECT * FROM products WHERE product_id = 12345;

1. Best Index: B-Tree

2. Why?

- **B-Tree** indexes are optimal for queries that involve equality or range searches on a single dimension. In this case, the search is filtering on the product_id, which is a single attribute.
- A **B-Tree** is well-suited for indexing discrete, ordered data (like product_id) and supports efficient equality searches.
- The B-Tree structure allows for fast lookup by narrowing down the search space through its ordered tree structure, making it highly efficient for equality queries.

3. Expected Performance Difference:

- A **B-Tree** index will provide **O(log N)** time complexity for searching for a specific product_id.
- **R-Tree** would be inefficient here since it is designed for multi-dimensional spatial data. Using an R-Tree would introduce unnecessary overhead.

Scenario B: Geographic 2D Search

Query:

SELECT * FROM stores

WHERE latitude BETWEEN 40.0 AND 41.0

AND longitude BETWEEN -74.0 AND -73.0;

1. Best Index: R-Tree

2. Why?

- **R-Tree** is specifically designed for **multi-dimensional spatial data**, such as geographic coordinates (latitude and longitude).

- It indexes data by bounding boxes (Minimum Bounding Rectangles, or MBRs), which works well for range queries that involve multiple dimensions, like this query which searches within a 2D geographical area.

- The **R-Tree** structure can efficiently process **spatial range queries** that are common in geographic or 2D data (e.g., finding stores within a rectangular area defined by latitude and longitude).

3. Expected Performance Difference:

- An **R-Tree** index will perform well with **O(log N)** complexity for spatial range queries, allowing fast retrieval of all stores within the specified geographic bounds.

- A **B-Tree** would not perform efficiently for multi-dimensional range queries like this, because it's optimized for single-dimensional searches. The B-Tree would require multiple scans or inefficient searches through all data entries, making it unsuitable for geographic searches.

Scenario C: Multi-Attribute Search

Query:

```
SELECT * FROM products

WHERE category_id = 5

AND price BETWEEN 100 AND 500;
```

1. Best Index: B-Tree

2. Why?

- **B-Tree** indexes are ideal for queries involving equality and range conditions on **multiple attributes**, as long as the attributes are **ordered**.

- In this case, the query involves filtering on two attributes: category_id (for exact matching) and price (for a range condition). A **B-Tree** can efficiently handle both conditions by creating a composite index that includes both category_id and price.

- Composite **B-Tree** indexes are commonly used for multi-attribute searches, allowing for efficient traversal through the index, minimizing the number of data rows that need to be scanned.

3. Expected Performance Difference:

- A **B-Tree** index on both category_id and price would enable the query to efficiently retrieve results with **O(log N)** time complexity for the range condition on price and direct matching for category_id.

- An **R-Tree** would not be optimal here because the query is not spatial in nature. R-Trees are specifically optimized for multi-dimensional spatial data, whereas B-Trees handle range and equality searches in ordered, single or composite attributes more efficiently.